

Manual de Uso

Equipo Enterprise de Agentes

22 agentes especializados · 2 clusters gobernados

Versión 1.0 **Fecha** 12 jul 2026 **Marca** DAG

Guía profesional y completa para saber cuándo, cómo y por qué invocar cada agente — tanto en un flujo de equipo end-to-end como de forma suelta en proyectos distintos.

Cómo leer este manual

Este documento está pensado para convertirse en PDF profesional. Está organizado en tres grandes bloques:

1. **Fundamentos** (secciones 1–4): filosofía, composición del equipo, principios de gobierno y los dos modelos de invocación.
2. **Referencia de agentes** (secciones 5–7): fichas individuales de los 22 agentes y explicación de los dos clusters.
3. **Operación práctica** (secciones 8–14): de la necesidad a la realidad, playbooks por escenario, RACI, handoffs, buenas prácticas, anti-patrones y plantillas.

Si solo quieres empezar rápido, ve a la **sección 8 (De la necesidad a la realidad)** y la **sección 15 (Cheat sheet)**.

Tabla de contenido

1. Filosofía del equipo
2. Composición del equipo
3. Principios de gobierno (léelos antes de invocar)
4. Los dos modelos de invocación
5. Fichas de agentes — Núcleo de producto y entrega
6. Cluster Data + IA
7. Cluster Seguridad (Destroyer)
8. De la necesidad a la realidad (flujo end-to-end)
9. Playbooks por escenario
10. Invocación suelta: usar un agente fuera del flujo
11. RACI y protocolo de handoff
12. Cómo sacarles el máximo provecho
13. Anti-patrones (qué NO hacer)
14. Plantillas listas para usar
15. Cheat sheet — referencia rápida

---

1. Filosofía del equipo

Este equipo no es una colección de asistentes genéricos: es una **organización de especialistas de nivel enterprise**, cada uno "Primer Maestro" o "Maestro Universal" de su disciplina. Su comportamiento se rige por cuatro principios que debes entender para explotarlos bien.

1.1 Neutralidad tecnológica total

Ningún agente favorece por defecto un stack, nube, framework o proveedor. Deciden **por contexto, evidencia, riesgo y costo**. Esto significa que puedes usarlos en cualquier proyecto (fintech, salud, e-commerce, govtech, IoT, gaming, etc.) sin que impongan una tecnología. Tú aportas el contexto; ellos aportan el criterio.

1.2 Ownership único (un solo dueño por decisión)

Cada decisión tiene **un único "Accountable" (A)**. Nunca hay "todos un poco responsables". Cuando dos agentes creen ser dueños de lo mismo, **gana el dueño del dominio** y el otro pasa a "Consultado". Esto evita solapamientos y parálisis.

1.3 Colaborar sí, reemplazar no

Cada agente es **A/R (Accountable/Responsible)** en su dominio y **C/I (Consultado/Informado)** en el de sus vecinos. Un agente de seguridad **define** el control; el de plataforma lo **opera**. Un agente de datos **diseña** la semántica; otro la **materializa**. Se pasan el trabajo con **handoffs explícitos**, no invadiendo territorio ajeno.

1.4 Modelo TIERED (no solo auditan: construyen)

Los agentes transversales (plataforma, performance, correctitud, documentación) no se limitan a recomendar: bajo su disciplina **implementan y corrigen** directamente lo cross-cutting que les pertenece (el pipeline, el módulo IaC, el perf gate, el runbook, la corrección de un bug de lógica).

> ***Consecuencia práctica:*** *obtienes lo mejor cuando le das a cada agente* ***su dominio y el contexto correcto****, y dejas que los cruces se resuelvan por handoff. Pedirle a un agente que decida fuera de su dominio degrada la calidad.*

---

2. Composición del equipo

22 agentes organizados en un núcleo de producto/entrega y dos clusters gobernados. Además hay **2 documentos de gobernanza de cluster** que definen ownership, RACI y handoffs.

2.1 Mapa general

Bloque	Agentes	Nº
Núcleo de producto y entrega	tompo, orquesta, eneida, uxuifull, farola, motor, palma, amado, certero, baluarte, peakformance, docpower	12
Cluster Data + IA	andulfo, caudal, brujula, prisma, sysuniversal-ai	5
Cluster Seguridad (Destroyer)	destroyer, cazador, centinela, sello, vector	5
Total		**22**

Documentos de cluster:

- `cluster-data-ai/gobernanza-y-colaboracion-cluster.md`
- `cluster-seguridad/gobernanza-y-colaboracion-cluster.md`

2.2 Mapa rol → agente

Rol en la organización	Agente(s)
Product Owner / BA (valor, prioridad, discovery)	`tompo`
Delivery Manager / Orquestador	`orquesta`
Arquitectura de software y solución	`eneida`
UX/UI (research, journeys, design system)	`uxuifull`
Frontend web	`farola`
Mobile (nativo/cross-platform)	`palma`
Backend (APIs, lógica de servidor)	`motor`
Plataforma / SRE / Entrega (CI/CD, IaC, SLO, HA/DR de infra)	`baluarte`
Performance	`peakformance`
Correctitud / code review / debugging	`certero`
QA / calidad	`amado`
Documentación	`docpower`
Datos: plataforma, pipelines, semántica	`andulfo` · `caudal` · `brujula`
Ciencia de datos (modelos, experimentos)	`prisma`
IA / LLM / MLOps	`sysuniversal-ai`
Seguridad	`destroyer` (+ `cazador` / `centinela` / `sello` / `vector`)

2.3 Diagrama de flujo de valor



3. Principios de gobierno (léelos antes de invocar)

Estos principios están codificados en los documentos de cluster y en las gobernanzas individuales. Respetarlos es lo que diferencia un uso amateur de uno profesional.

1. **Un único A por decisión.** Antes de invocar, pregúntate: *¿de quién es esta decisión?* Enruta al dueño correcto.
2. **La costura se resuelve con handoff, no con invasión.** Si tu tarea cruza dominios, usa el paquete de handoff (sección 11 y 14).
3. **Los gates no se saltan.** Correctitud (`certero`), calidad (`amado`) y seguridad (`destroyer` + cluster) son puertas de calidad; intégralas, no las omitas.

4. **Seguridad ofensiva requiere autorización (RoE) firmada.** `cazador` y cualquier actividad ofensiva **nunca** actúan sin alcance y autorización escritos.
5. **Nada crítico va a producción sin dueño de operación.** Un modelo de IA sin `sysuniversal-ai` (MLOps) o un servicio sin `baluarte` (SRE) es un anti-patrón.
6. **El significado de negocio lo decide `tompo`; la implementación técnica, el especialista.** Una métrica tiene doble dueño: `tompo` (qué significa) y `brujula` (cómo se calcula).
7. `orquesta` **coordina, no decide el contenido.** Orquesta secuencia y desatasca; nunca sobrescribe el valor (tompo), la arquitectura (eneida), el pipeline (baluarte) ni el craft del especialista.

---

4. Los dos modelos de invocación

Hay exactamente dos formas de usar este equipo. Entender la diferencia es la clave para sacarle provecho.

4.1 Modelo A — Flujo orquestado (equipo completo)

Para **iniciativas** (una feature, un producto, una migración, un incidente mayor). El trabajo cruza varios dominios y necesita secuencia.

- **Puerta de entrada:** `tompo` (si la duda es *qué/por qué*) o `orquesta` (si ya sabes *qué* y necesitas coordinar el *cómo/cuándo/quién*).
- **Orquesta es el "front door" de lo multi-disciplina:** clasifica la señal (valor→tompo, técnica→eneida/especialista, flujo→orquesta), la descompone y devuelve el mapa señal→dueño con su secuencia.
- Sigue la **sección 8** para el flujo completo.

4.2 Modelo B — Invocación suelta (un agente puntual)

Para una **tarea de un solo dominio** en cualquier proyecto, aunque no sea parte de un flujo del equipo. Ejemplos: "revisa la seguridad de este endpoint" → `destroyer`; "optimiza esta consulta" → `andulfo` o `peakformance`; "audita la accesibilidad de esta pantalla" → `uxuifull`.

- **Puerta de entrada:** el agente del dominio, directamente.
- **No necesitas** pasar por `tompo` ni `orquesta` si la tarea es de un solo dominio y no dispara dependencias.
- La **sección 10** detalla cómo invocar cada agente de forma suelta.

> ***Regla de decisión:*** *¿La tarea cruza más de un dominio o crea dependencias entre agentes?* ***Sí*** *→ Modelo A (orquestado).* ***No*** *→ Modelo B (suelto).*

---

5. Fichas de agentes — Núcleo de producto y entrega

Cada ficha incluye: **rol**, **cuándo invocar**, **triggers** (palabras/situaciones gatillo), **qué entrega**, **uso suelto** (fuera del flujo del equipo) y **qué NO hace** (para no confundir dueños).

5.1 `tompo` — Product Owner / Business Analyst

- **Rol:** Maximiza el valor del producto y gobierna el backlog. Dueño del *qué* y el *por qué*.

- **Cuándo invocar:** al inicio de cualquier iniciativa, cuando hay que priorizar, definir alcance de valor, escribir historias de usuario o decidir qué construir primero.
- **Triggers:** "¿qué construimos?", priorización, backlog, roadmap, discovery, historia de usuario, criterios de aceptación, OKR, North Star, MVP.
- **Entrega:** Product Goal, backlog priorizado (RICE/WSJF/MoSCoW/Kano), historias INVEST con criterios de aceptación (Gherkin/BDD), DoR/DoD, métricas de producto.
- **Uso suelto:** priorizar un backlog existente; escribir criterios de aceptación de una feature aislada; definir métricas de éxito de un experimento; hacer discovery de una idea.
- **NO hace:** no decide arquitectura (eneida), no coordina la ejecución (orquesta), no implementa.

5.2 `orquesta` — Delivery Manager / Orquestador

- **Rol:** Único agente *meta* del equipo. Coordina *quién hace qué, en qué orden, con qué dependencias y capacidad*. Es el "front door" de lo multi-disciplina.
- **Cuándo invocar:** cuando una iniciativa cruza varios agentes, hay dependencias, riesgos (RAID), necesitas un plan de entrega, o el flujo se atasca.
- **Triggers:** "coordina", plan de entrega, dependencias, secuencia, capacidad, cadencia, estado, escalamiento, war room.
- **Entrega:** plan secuenciado con dependencias, ventana de release, gestión de RAID, reportes de estado, handoffs a los dueños.
- **Uso suelto:** planificar la ejecución de un proyecto multi-tarea; desatascar un flujo bloqueado; coordinar un incidente entre varios responsables.
- **NO hace:** no sobrescribe valor (tompo), arquitectura (eneida), pipeline (baluarte) ni el craft de un dominio. Coordina; no decide el contenido.

5.3 `eneida` — Arquitecto de Software y de Solución

- **Rol:** Dueño del *cómo técnico*: estilo arquitectónico, patrones, contratos de API/eventos, modelo de datos, decisiones tecnológicas, atributos de calidad y SLO objetivo.
- **Cuándo invocar:** al diseñar un sistema nuevo, evaluar una decisión técnica de peso, definir contratos entre servicios, o revisar la arquitectura de algo existente.
- **Triggers:** arquitectura, diseño técnico, ADR, patrón, contrato de API, trade-offs, selección de tecnología, atributos de calidad, escalabilidad.
- **Entrega:** diseño de arquitectura con alternativas y trade-offs, ADRs, contratos (OpenAPI/AsyncAPI/DDL), invariantes y handoffs a especialistas.
- **Uso suelto:** revisar la arquitectura de un proyecto ajeno; producir un ADR para una decisión concreta; diseñar el contrato de una API nueva.
- **NO hace:** no implementa la feature (motor/farola/palma), no prioriza valor (tompo), no opera la plataforma (baluarte).

5.4 `uxuifull` — UX/UI Integral

- **Rol:** Diseñador y estratega de experiencia end-to-end: UX research, arquitectura de información, interaction design, UI/visual, design systems, accesibilidad (WCAG 2.2/EAA), UX writing, service design.
- **Cuándo invocar:** antes de construir UI, para diseñar flujos, auditar usabilidad/accesibilidad, crear o gobernar un design system, o redactar microcopy.
- **Triggers:** UX, UI, research, journey, wireframe, design system, accesibilidad, WCAG, usabilidad, microcopy, prototipo.

- **Entrega:** flujos e IA, wireframes/prototipos, design system con tokens, auditorías de usabilidad/accesibilidad, especificaciones de interacción.
- **Uso suelto:** auditar la accesibilidad de una pantalla; rediseñar un flujo confuso; crear un design system desde cero; mejorar el copy de una UI.
- **NO hace:** no es dev ni arquitecto; no decide prioridades ni lidera técnicamente.

5.5 `farola` — Frontend Engineering (web)

- **Rol:** Arquitecto y principal frontend engineer web (responsive, PWA, UI web).
- **Cuándo invocar:** para construir o mejorar interfaces web, arquitectura de frontend, rendimiento percibido, componentes.
- **Triggers:** frontend, web, React/Vue/Svelte/Angular (neutral), componente, estado, SSR/SSG, PWA, bundle, responsive.
- **Entrega:** implementación de UI web, arquitectura de frontend, integración con APIs, optimización de carga percibida.
- **Uso suelto:** construir un componente/página; refactorizar un frontend; mejorar Core Web Vitals de un sitio.
- **NO hace:** no diseña la experiencia (uxuifull), no construye el backend (motor), no hace mobile nativo (palma).

5.6 `palma` — Ingeniería Mobile

- **Rol:** Arquitecto e ingeniero mobile nativo/cross-platform, incluido el ciclo de tiendas.
- **Cuándo invocar:** para construir apps móviles, decidir nativo vs cross-platform, gestionar publicación en tiendas, o rendimiento móvil.
- **Triggers:** mobile, iOS, Android, nativo, cross-platform (neutral), tienda/App Store/Play, push, offline-first.
- **Entrega:** apps móviles, arquitectura mobile, integración con backend, estrategia de publicación y actualización.
- **Uso suelto:** construir una app; auditar una app existente; planear la subida a tiendas.
- **NO hace:** no hace web (farola), no diseña la experiencia (uxuifull).

5.7 `motor` — Backend Prime Developer

- **Rol:** Desarrollador backend maestro: APIs, lógica de servidor, integraciones, persistencia a nivel de aplicación.
- **Cuándo invocar:** para implementar servicios, endpoints, lógica de negocio, integraciones y toda la construcción del lado servidor.
- **Triggers:** backend, API, endpoint, servicio, microservicio, lógica de negocio, integración, cola, worker.
- **Entrega:** servicios backend, APIs implementadas, integraciones, lógica de negocio, según contratos de eneida.
- **Uso suelto:** implementar un endpoint; refactorizar un servicio; integrar un tercero; construir un worker.
- **NO hace:** no diseña la arquitectura global (eneida), no es dueño de la plataforma de datos (andulfo), no opera infraestructura (baluarte).

5.8 `baluarte` — Plataforma, Entrega y Confiabilidad (DevOps + SRE + Platform)

- **Rol:** Dueño del *cómo* se construye-empaqueta-despliega-opera y se mantiene confiable: CI/CD, IaC, contenedores/orquestación, observabilidad, SLI/SLO/error budgets, incidentes, HA/DR de

infraestructura, FinOps.

- **Cuándo invocar:** para pipelines, despliegues, infraestructura como código, definición de SLOs, gestión de incidentes operativos, o mejorar confiabilidad.
- **Triggers:** CI/CD, pipeline, deploy, IaC, Kubernetes/contenedores (neutral), SLO, error budget, observabilidad, on-call, incidente, rollback, canary/blue-green.
- **Entrega:** pipelines, módulos IaC, golden paths, definición de SLOs y políticas de error budget, observabilidad/alertas, runbooks, proceso de incidentes.
- **Uso suelto:** montar un pipeline CI/CD; escribir IaC; definir SLOs; investigar un incidente de plataforma; optimizar costos cloud.
- **NO hace:** no decide el control de seguridad (lo define el cluster; baluarte lo *opera*); no diseña la arquitectura de aplicación (eneida); no es dueño de la plataforma de *datos* (andulfo).

5.9 `peakformance` — Performance Engineering

- **Rol:** Dueño de metas/budgets/profiling de rendimiento del ecosistema, y bajo modelo TIERED corrige lo cross-cutting de performance.
- **Cuándo invocar:** cuando hay problemas de latencia/throughput/costo, para fijar performance budgets, perfilar, o antes de un lanzamiento con exigencias de rendimiento.
- **Triggers:** performance, latencia, throughput, profiling, budget de rendimiento, cuello de botella, optimización, carga, escalabilidad de rendimiento.
- **Entrega:** metas y budgets de performance, análisis de profiling, perf gates, harnesses de medición, correcciones cross-cutting de rendimiento.
- **Uso suelto:** perfilar y optimizar una app lenta; fijar budgets de rendimiento; diseñar pruebas de carga.
- **NO hace:** no implementa la feature funcional; el tuning específico de datos lo ejecuta el cluster (andulfo/caudal/AI) dentro de las metas que peakformance fija.

5.10 `certero` — Code Review & Debugging

- **Rol:** Revisor de código y cazador/corrector de defectos. Dueño de la **correctitud**: bugs de lógica, edge cases, manejo de errores, invariantes.
- **Cuándo invocar:** para revisar código, cazar un bug difícil, validar correctitud antes de merge, o hacer debugging sistemático.
- **Triggers:** code review, bug, debug, defecto, edge case, regresión, correctitud, "no funciona", revisión de PR.
- **Entrega:** revisión de código con severidad, diagnóstico de causa raíz, correcciones de defectos, validación de invariantes.
- **Uso suelto:** revisar un PR de cualquier proyecto; depurar un bug puntual; auditar la correctitud de un módulo.
- **NO hace:** no diseña arquitectura (eneida), no define estrategia de pruebas (amado); cuida que el código haga bien lo diseñado.

5.11 `amado` — QA / Calidad

- **Rol:** Primer Maestro de Calidad: estrategia y arquitectura de calidad, planes de prueba, diseño de casos, automatización en CI/CD, risk-based testing, pruebas no funcionales, evaluación de sistemas de IA.
- **Cuándo invocar:** para definir la estrategia de pruebas, diseñar casos, montar automatización, planear regresión/smoke/sanity, o auditar la calidad de una implementación.

- **Triggers:** QA, testing, plan de pruebas, casos de prueba, automatización, cobertura, regresión, calidad, evals de IA.
- **Entrega:** estrategia y plan de pruebas (ISTQB/ISO 29119), casos de prueba, frameworks de automatización, informes de calidad, gates de calidad.
- **Uso suelto:** crear un plan de pruebas; diseñar casos para una feature; montar un framework de automatización; auditar la calidad de un sistema.
- **NO hace:** no corrige el código (certero), no implementa la feature.

5.12 `docpower` — Documentation Engineering

- **Rol:** Technical Writer / Documentation Engineer senior. Dueño de la **forma** documental: estructura, estándares editoriales, discoverability, diagramas.
- **Cuándo invocar:** para diseñar y producir documentación (técnica, de API, de usuario, ADRs, runbooks), estandarizar docs, o mejorar la documentación existente.
- **Triggers:** documentación, docs, README, guía, manual, API reference, runbook, changelog, diccionario de datos.
- **Entrega:** documentación estructurada y publicable, estándares editoriales, arquitectura de información documental.
- **Uso suelto:** documentar un proyecto; escribir un README/guía; estandarizar la documentación de un repo.
- **NO hace:** no genera el contenido técnico de fondo (lo aporta el especialista); gobierna la forma, no el fondo.

---

6. Cluster Data + IA

> ***Documento de gobierno:*** `cluster-data-ai/gobernanza-y-colaboracion-cluster.md`
> ***Principio rector:*** *colaborar sí, reemplazar no. Cada agente es A/R en su dominio y C/I en el vecino.*

6.1 Cadena de valor



El valor fluye del **dato en reposo** al **sistema de IA operado**. Cada eslabón consume lo del anterior y entrega una base más lista para uso.

6.2 Fichas

`ANDULFO` — DBA / ARQUITECTO DE PLATAFORMA DE DATOS

- **A/R de:** plataforma, motores, topología, HA/DR, backups, seguridad y gobierno de persistencia, cambios estructurales de esquema base.
- **Invócalo para:** elegir motor, diseñar/revisar esquemas, optimizar consultas y planes, planear HA/DR y backup/restore, capacity planning, gobierno del dato.
- **Uso suelto:** tuning de una base de datos; diseño de un esquema; plan de HA/DR; auditoría de una plataforma de datos.

`CAUDAL` — DATA ENGINEERING / PIPELINES

- **A/R de:** ingestión (CDC/streaming), ETL/ELT, orquestación, contratos y calidad técnica del dato en movimiento; materialización de datasets (`staging → core`).
- **Invócalo para:** diseñar/construir pipelines, ingesta multi-fuente, streaming, orquestación, calidad de datos, DataOps.
- **Uso suelto:** construir un pipeline de ingesta; montar streaming; migrar datos entre sistemas.

`BRUJULA` — ANALYTICS ENGINEERING / CAPA SEMÁNTICA

- **A/R de:** modelos semánticos (dimensiones/hechos/OBT) y las **definiciones técnicas** de métricas/KPIs oficiales; datasets "listos para consumo".
- **Invócalo para:** modelar la capa semántica, definir/gobernar métricas ("una métrica, una definición"), crear marts/vistas de consumo, tests de calidad de la capa.
- **Uso suelto:** definir métricas oficiales; construir un modelo dimensional; crear un diccionario de métricas.

`PRISMA` — DATA SCIENCE

- **A/R de:** features (diseño/lógica), selección y validación metodológica de modelos, experimentación, insights.
- **Invócalo para:** EDA, modelado (regresión, clasificación, clustering, series de tiempo, causal), experimentos A/B, feature engineering, insights accionables.
- **Uso suelto:** análisis exploratorio de un dataset; construir un modelo predictivo; diseñar un experimento; extraer insights.

`SYSUNIVERSAL-AI` — AI ENGINEERING / LLM / MLOPS

- **A/R de:** aprobación a producción, serving, MLOps/LLMOps (deploy, monitoreo, drift, retraining, rollback), guardrails, orquestación de LLMs/agentes/RAG.
- **Invócalo para:** llevar modelos a producción, arquitectura de sistemas de IA (RAG, agentes, inferencia), lifecycle de modelos, performance de inferencia.
- **Uso suelto:** diseñar un sistema RAG; poner un modelo en producción con monitoreo; construir un agente con tools; optimizar inferencia.

6.3 Reglas de oro del cluster Data + IA

- El **significado** de una métrica es de `tompo` ; su **implementación técnica**, de `brujula` .
- `prisma` **no** toca motor ni HA/DR: solicita cambios a nivel features/datasets; lo base va por `andulfo` .
- `sysuniversal-ai` **construye y opera** la IA; `vector` (cluster seguridad) la **asegura**.
- Ningún modelo crítico entra a producción sin `sysuniversal-ai` como dueño del lifecycle.

---

7. Cluster Seguridad (Destroyer)

> ***Documento de gobierno:*** `cluster-seguridad/gobernanza-y-colaboracion-cluster.md`
> ***Línea roja no negociable:*** *toda actividad ofensiva exige* ***RoE (autorización) firmado****. Ningún agente puede relajarla.*

7.1 Cadena de valor

```
destroyer (núcleo) → cazador → [remediación] → centinela → sello
                                    └── vector (transversal cuando hay
IA/LLM/agentes)
```

El valor fluye del **diseño seguro** a la **operación gobernada y auditable**.

7.2 Fichas

`DESTROYER` — SECURITY ARCHITECT + DEVSECOPS + ORQUESTADOR DEL CLUSTER (NÚCLEO)

- **A/R de:** arquitectura segura, threat model, diseño de controles, gates de SDLC/CI-CD, gestión de secretos y criptografía, remediación de ingeniería, sign-off técnico de seguridad.
- **Puerta de entrada del cluster:** es el front door transversal de seguridad; enruta a los satélites cuando el dominio es explícito.
- **Invócalo para:** diseñar seguridad *by design*, modelar amenazas, definir controles y gates, gestionar secretos/cripto, remediar hallazgos.
- **Uso suelto:** threat model de un sistema; diseñar controles de seguridad; revisar la seguridad de un diseño; gestionar secretos.

`CAZADOR` — SEGURIDAD OFENSIVA (RED TEAM / PENTEST)

- **A/R de:** ataque autorizado (recon, enumeración, explotabilidad, chaining, PoC controlada) y retest que confirma el cierre.
- **Requiere:** RoE firmado. **Sin autorización escrita, no hay pentest.**
- **Invócalo para:** pentest, red team, análisis de explotabilidad, PoC controlada, retest de remediaciones.
- **Uso suelto:** pentest de una app/API (con RoE); validar la explotabilidad de una vulnerabilidad; retest tras un fix.
- **NO hace:** no remedia (destroyer + dueño), no opera detección (centinela), no despliega controles.

`CENTINELA` — SEGURIDAD DEFENSIVA (BLUE TEAM / SECOPS / DFIR)

- **A/R de:** hardening operativo, detection engineering, SOC/monitoreo, threat hunting, respuesta a incidentes y forense (DFIR/IR).
- **Invócalo para:** construir detección (SIEM/EDR), hardening, monitoreo, y conducir la respuesta a un incidente.
- **Uso suelto:** montar detección/alertas; endurecer un sistema; investigar un incidente; threat hunting.
- **NO hace:** no ejecuta ataques (cazador), no diseña la arquitectura de controles (destroyer).

`SELLO` — GRC + PRIVACIDAD + AUDITORÍA

- **A/R de:** riesgo formal, políticas, ISO 27001/27701/42001, auditoría, privacidad técnica/DPIA/DPO, factor humano, terceros (TPRM).

- **Invócalo para:** medir/priorizar riesgo, definir políticas, auditar contra marcos, gobernar privacidad (DPIA, Ley 1581/GDPR), programas de concienciación.
- **Uso suelto:** auditoría ISO/NIST; DPIA de un tratamiento de datos; política de seguridad; evaluación de riesgo.
- **NO hace:** no diseña controles técnicos (destroyer), no interpreta la ley (legal), no opera detección (centinela).

`VECTOR` — SEGURIDAD DE IA/LLM/AGENTES (MLSECOPS, TRANSVERSAL)

- **A/R de:** seguridad específica de IA: prompt injection (directa/indirecta), excessive agency, abuso de herramientas, fuga de datos en contexto, envenenamiento de datos/RAG, guardrails (OWASP LLM Top 10 + MITRE ATLAS).
- **Invócalo para:** asegurar cualquier sistema con IA/LLM/agentes; diseñar guardrails y evals de seguridad de IA.
- **Uso suelto:** revisar un agente/LLM por prompt injection; diseñar guardrails; red-teaming de IA (defensivo).
- **Naturaleza:** transversal — cuando **no** hay IA, vector no participa; cuando el control es de IA, es el dueño y los demás pasan a C.

7.3 Reglas de oro del cluster Seguridad

- **Nadie hace ofensiva sin RoE firmado.**
- **Ningún hallazgo se cierra sin retest** (fix por destroyer+dueño, retest por cazador).
- `sello` gobierna y audita; **no** diseña controles técnicos.
- El cluster **define** el control; `baluarte` lo **opera** (pipeline, secretos, contención). En un incidente, seguridad dirige y baluarte ejecuta la restauración.
- Sistemas con impacto regulatorio o PII: el sign-off técnico (destroyer) coexiste con el de negocio (tompo), cumplimiento (sello) y legal.

---

8. De la necesidad a la realidad (flujo end-to-end)

Este es el proceso recomendado para convertir una necesidad en algo entregado y operado. Adáptalo al tamaño del trabajo (para tareas de un solo dominio, salta al Modelo B / sección 10).

Fase 0 — Formula la necesidad (tú)

Antes de invocar, escribe un **brief de necesidad** (plantilla en sección 14.1): problema, resultado esperado, usuarios/impacto, restricciones, criterios de éxito. Un buen brief multiplica la calidad de todo lo que sigue.

Fase 1 — Valor y alcance → `tompo`

Entrega el brief a `tompo`. Define el Product Goal, prioriza, escribe historias con criterios de aceptación y DoR/DoD. **Salida:** backlog priorizado con el *qué* y el *por qué* claros.

Fase 2 — Arquitectura y factibilidad → `eneida`

`eneida` diseña la solución con **alternativas y trade-offs**, produce ADRs y contratos (API/eventos/datos), fija atributos de calidad y SLO objetivo. Consulta al cluster de datos/IA y a seguridad para factibilidad. **Salida:** diseño + ADRs + contratos + invariantes.

Fase 3 — Plan de ejecución → `orquesta`

`orquesta` toma valor (tompo) + arquitectura (eneida), descompone en tareas, define secuencia, dependencias (RAID) y ventana. Reparte handoffs a los dueños. **Salida:** plan secuenciado y asignado.

Fase 4 — Diseño de experiencia (si hay UI) → `uxuifull`

Flujos, IA, wireframes/prototipos, design system y accesibilidad **antes** de construir la UI. **Salida:** especificación de diseño lista para implementar.

Fase 5 — Construcción (en paralelo según dependencias)

- Backend → `motor` · Frontend web → `farola` · Mobile → `palma`
- Datos → `andulfo` (plataforma) · `caudal` (pipelines) · `brujula` (semántica)
- Ciencia de datos → `prisma` · IA en producción → `sysuniversal-ai`

Cada especialista construye según los contratos de `eneida` y la prioridad de `tompo`.

Fase 6 — Gates de calidad transversales (siempre)

- **Correctitud** → `certero` (revisión de código, debugging)
- **Calidad** → `amado` (estrategia y ejecución de pruebas, automatización)
- **Performance** → `peakformance` (budgets, profiling)
- **Seguridad** → `destroyer` (+ `cazador` / `centinela` / `sello` / `vector` según aplique)

Estos gates se integran en la cadencia; no son un paso final opcional.

Fase 7 — Entrega y operación → `baluarte`

Empaqueta, despliega (canary/blue-green/rolling), define SLOs y error budgets, observabilidad, runbooks y on-call. **Salida:** sistema en producción, confiable y observable.

Fase 8 — Documentación y cierre → `docpower`

Documentación técnica, de API, de usuario, ADRs y runbooks, con forma estandarizada y trazable. **Salida:** conocimiento capturado y descubrible.

Fase 9 — Retro y mejora continua → `tompo` + `orquesta`

Mide contra los criterios de éxito, recoge aprendizajes, realimenta el backlog.

> ***Regla de flujo:*** *cada transición entre fases es un* ***handoff explícito*** *(sección 11). Lo que no se entrega con contexto, se pierde en la costura.*

---

9. Playbooks por escenario

Recetas concretas para situaciones frecuentes. Indican el orden de invocación y el dueño de cada paso.

9.1 Nueva feature en un producto existente

`tompo` (valor, historia, AC) → `eneida` (impacto arquitectónico, contrato) → `orquesta` (plan) → `uxuifull` (si hay UI) → `motor` / `farola` / `palma` (construcción) → `certero` + `amado` (gates) → `destroyer` (si toca datos sensibles/superficie de ataque) → `baluarte` (deploy) → `docpower` (docs).

9.2 Bug en producción

`certero` (diagnóstico de causa raíz + fix) → `amado` (test de regresión) → `baluarte` (despliegue del fix / rollback si urge) → `orquesta` (coordina si cruza equipos) → `docpower` (postmortem si aplica). Si es incidente de **seguridad**: `centinela` dirige (IR/DFIR) y `baluarte` ejecuta contención/restauración.

9.3 Auditoría de seguridad de un sistema

`destroyer` (threat model + revisión de arquitectura) → `cazador` (pentest, **con RoE firmado**) → `destroyer` + dueño (remediación) → `cazador` (retest) → `centinela` (detección/hardening) → `sello` (riesgo residual, cumplimiento). Si hay IA/LLM: añade `vector`.

9.4 Migración o nueva plataforma de datos

`eneida` (arquitectura) → `andulfo` (plataforma, motor, HA/DR) → `caudal` (pipelines/migración de datos) → `brujula` (semántica/métricas sobre el nuevo core) → `amado` (validación de datos) → `baluarte` (infraestructura) → `docpower` (diccionario/linaje).

9.5 Poner un modelo/sistema de IA en producción

`tompo` (caso de uso, valor) → `prisma` (modelo, validación metodológica) → `sysuniversal-ai` (serving, MLOps/LLMOps, guardrails) → `vector` (seguridad de IA) → `sello` (riesgo/ISO 42001/privacidad) → `sysuniversal-ai` (monitoreo/drift en operación) → `baluarte` (infraestructura de serving).

9.6 Mejorar el rendimiento de una aplicación

`peakformance` (perfilar, fijar budgets, identificar cuellos) → especialista del dominio del cuello (`andulfo` / `caudal` para datos, `motor` para backend, `farola` / `palma` para cliente, `sysuniversal-ai` para inferencia) ejecuta el tuning dentro de las metas → `amado` (pruebas de carga) → `baluarte` (autoscaling/observabilidad).

9.7 Rediseño de UX de un flujo confuso

`uxuifull` (research, rediseño de flujo, accesibilidad) → `tompo` (validar valor/prioridad) → `farola` / `palma` (implementación) → `amado` (pruebas de usabilidad/accesibilidad) → `docpower` (guías).

9.8 Nuevo producto desde cero (greenfield)

`tompo` (discovery, Product Goal) → `eneida` (arquitectura fundacional) → `uxuifull` (experiencia) → `orquesta` (plan) → construcción (`motor` / `farola` / `palma` /cluster datos) → gates (`certero` / `amado` / `peakformance` / `destroyer`) → `baluarte` (plataforma y entrega) → `docpower`.

9.9 Auditoría de calidad de un sistema heredado

`amado` (estrategia y auditoría de calidad) + `certero` (revisión de correctitud) + `peakformance` (rendimiento) + `destroyer` (postura de seguridad) → `eneida` (recomendaciones arquitectónicas) → `tompo` (priorizar remediaciones) → `orquesta` (plan de saneamiento).

9.10 Cumplimiento / privacidad (GDPR, Ley 1581/Habeas Data)

`sello` (DPIA, gap analysis, políticas) → `destroyer` (controles técnicos de privacidad/cifrado) → `andulfo` (gobierno del dato) → `eneida` (privacidad by design en arquitectura) → `amado` (verificación) → `docpower` (evidencia documental).

---

10. Invocación suelta: usar un agente fuera del flujo

No siempre necesitas al equipo completo. Muchas veces quieres **un especialista para una tarea puntual** en cualquier proyecto. Esto es totalmente válido y es donde estos agentes brillan como consultores de élite.

10.1 Cuándo usar invocación suelta

- La tarea es de **un solo dominio**.
- No genera dependencias que otro agente deba coordinar.
- Quieres una opinión/entrega experta rápida.

10.2 Cómo invocar un agente suelto (buenas prácticas)

1. **Nombra el agente y su dominio explícitamente.** Ej.: "Actúa como `andulfo` y optimiza esta consulta".
2. **Da contexto mínimo suficiente:** objetivo, restricciones, entorno, criterio de éxito.
3. **Delimita el alcance:** deja claro que es una tarea puntual, no una iniciativa.
4. **Respeta la línea roja:** para `cazador` /ofensiva, incluye la autorización (RoE).

10.3 Catálogo rápido de invocación suelta

Necesito…	Invoca
Priorizar un backlog / escribir historias	`tompo`
Un plan de ejecución de un proyecto multi-tarea	`orquesta`
Un ADR o revisar una arquitectura	`eneida`
Auditar accesibilidad / rediseñar un flujo	`uxuifull`
Construir/refactorizar UI web	`farola`
Construir/auditar una app móvil	`palma`
Implementar/refactorizar un endpoint o servicio	`motor`
Montar CI/CD, IaC, definir SLOs	`baluarte`
Perfilar y optimizar rendimiento	`peakformance`
Revisar un PR / cazar un bug	`certero`
Plan de pruebas / automatización	`amado`
Documentar un proyecto / API	`docpower`
Tuning de BD / esquema / HA-DR	`andulfo`
Construir un pipeline / migrar datos	`caudal`
Definir métricas / modelo dimensional	`brujula`
Análisis de datos / modelo predictivo	`prisma`
Sistema RAG / agente / modelo a producción	`sysuniversal-ai`
Threat model / diseño de controles / secretos	`destroyer`
Pentest / red team (con RoE)	`cazador`
Detección / hardening / respuesta a incidente	`centinela`
Auditoría ISO / DPIA / riesgo	`sello`
Seguridad de IA/LLM/agentes	`vector`

> ***Tip pro:*** *aunque invoques suelto, el agente te dirá si la tarea realmente cruza dominios y conviene escalar a* `orquesta`*. Escúchalo: es la señal de que estás ante una iniciativa, no una tarea.*

---

11. RACI y protocolo de handoff

11.1 RACI de arquitectura (decisiones típicas)

Leyenda: **R** Responsable · **A** Aprobador (dueño) · **C** Consultado · **I** Informado.

Decisión	eneida	tompo	orquesta	baluarte	Especialista	Seguridad
Estilo arquitectónico / patrón	R/A	C	I	C	C	C
Contrato de API / evento	R/A	C	I	I	C	C
Modelo de datos / consistencia	R/A	I	I	C	C (datos)	C
Selección de tecnología	R	C	C	C	C	C
Atributos de calidad y SLO	R	C	I	A (operación)	C	C
Priorización del backlog	C	R/A	C	I	I	I
Alcance y fecha de release	I	A	R	C	C	C
Operación / on-call / runbooks	C	I	I	R/A	C	C
Implementación de una historia	A (diseño)	I	I	I	R	C
Postura de seguridad / cumplimiento	C	I	I	C	C	R/A

> *El "Especialista" es el dueño del dominio:* `farola` / `uxuifull` / `palma` / `motor` */cluster datos/* `peakformance` / `certero` / `amado` / `docpower`. *"Seguridad" =* `destroyer` *(+ cluster).*

11.2 Regla anti-conflicto

Si dos agentes creen ser **A** de la misma decisión, **gana el dueño del dominio** y el otro pasa a C. Si la decisión cruza dominios sin A claro, se **escala a** `orquesta` (para el flujo) o al dueño correspondiente (para el contenido).

11.3 Paquete de handoff (el corazón de la colaboración)

Cuando delegas o cambias de fase, entrega esto (no ejecutes por el otro dominio):

```
## Handoff → [agente/rol dueño]
- Tarea: [qué se transfiere y por qué es de este dueño]
- Outcome esperado: [resultado y criterio de "hecho"]
- Contexto: [problema, objetivo de negocio, estado actual]
- Prioridad de valor (tompo): [dónde entra en el orden]
- Restricciones técnicas (eneida): [invariantes/contratos que NO se rompen]
- Dependencias: [de qué depende / qué depende de esto; ruta crítica]
- Fecha objetivo / ventana: [rango + confianza]
- Gates a integrar: [correctitud=certero, calidad=amado, seguridad=destroyer,
release=baluarte]
- Lo que decide el dueño: [explícito]
- Validación de regreso: [cómo se verificará que se respetó lo acordado]
```

12. Cómo sacarles el máximo provecho

1. **Empieza por el brief, no por el agente.** Un problema bien formulado (sección 14.1) es el 50% del resultado.
2. **Enruta a la puerta correcta.** Iniciativa multi-dominio → `tompo` / `orquesta`. Tarea de un dominio → el especialista directo.
3. **Deja que cada agente sea dueño de su dominio.** No pidas a `motor` decisiones de arquitectura ni a `sello` que diseñe controles técnicos: baja la calidad y rompe el gobierno.
4. **Exige alternativas y trade-offs.** Especialmente a `eneida`: nunca aceptes una sola opción sin comparación.
5. **Integra los gates temprano.** Correctitud, calidad, performance y seguridad como parte de la cadencia, no al final.
6. **Usa handoffs explícitos entre fases.** La mayor pérdida de calidad ocurre en las costuras entre dominios.
7. **Aprovecha la neutralidad tecnológica.** Diles tu contexto real (stack, restricciones, presupuesto) para que decidan bien; no asumas que conocen tu entorno.
8. **Para seguridad ofensiva, prepara el RoE.** Sin autorización escrita, `cazador` no actúa (correctamente).
9. **Cierra el ciclo con `docpower`.** El conocimiento no documentado se evapora; deja trazabilidad.
10. **Reutiliza los clusters como unidad.** Para datos/IA o seguridad, invoca el cluster y deja que su gobernanza interna reparta el trabajo.
11. **Escala cuando el agente lo sugiera.** Si un especialista te dice "esto cruza dominios", conviértelo en iniciativa con `orquesta`.
12. **Mide contra los criterios de éxito de `tompo`.** Cierra cada iniciativa comparando resultado vs. objetivo.

---

13. Anti-patrones (qué NO hacer)

- **Ownership difuso:** "que todos revisen". Rompe el principio de A único. Asigna un dueño.
- **Saltarte los gates:** desplegar sin correctitud (`certero`), calidad (`amado`) o seguridad (`destroyer`).
- **Pedir a un agente fuera de su dominio:** ej. a `caudal` que defina métricas de negocio, o a `sello` que implemente controles.
- **Ofensiva sin RoE:** pedir un pentest sin autorización firmada.
- **Cerrar un hallazgo sin retest:** reportar y "arreglar" sin que `cazador` confirme el cierre.
- **Modelo de IA en producción sin dueño de MLOps** (`sysuniversal-ai`) o servicio sin SRE (`baluarte`).
- **Usar `orquesta` como embudo:** rutear a orquesta tareas de un solo dominio que van directo al especialista.
- **Confundir "define el control" con "opera el control":** el cluster de seguridad define; `baluarte` opera.
- **Empezar a construir sin arquitectura ni contratos** (`eneida`) cuando la iniciativa lo amerita.
- **Documentar al final y a la carrera:** integra `docpower` como parte del flujo.

---

14. Plantillas listas para usar

14.1 Brief de necesidad (para arrancar cualquier trabajo)

```
# Brief de necesidad
- Problema / oportunidad: [qué duele o qué queremos lograr]
- Resultado esperado (outcome): [cómo se ve el éxito]
- Usuarios / impacto: [a quién afecta y cuánto]
- Contexto / entorno: [stack, sistemas actuales, restricciones]
- Restricciones: [tiempo, presupuesto, legales/regulatorias, técnicas]
- Criterios de éxito (medibles): [métricas, umbrales]
- Riesgos conocidos: [lo que ya sabemos que puede fallar]
- ¿Un dominio o varios?: [define si es tarea suelta o iniciativa]
```

14.2 Señal entrante (para rutear a `orquesta`)

```
# Señal → orquesta
- Qué observo: [la necesidad/problema en una frase]
- Por qué creo que cruza dominios: [dependencias visibles]
- Urgencia / ventana: [cuándo se necesita]
- Dueño(s) que sospecho: [tompo/eneida/especialista/seguridad]
```

14.3 Solicitud de invocación suelta (para un especialista)

```
# Tarea puntual → [agente]
- Objetivo: [qué necesito exactamente]
- Contexto mínimo: [entorno, stack, restricciones]
- Alcance: [qué SÍ y qué NO entra]
- Criterio de "hecho": [cómo sé que quedó bien]
- Autorización (solo ofensiva): [RoE, alcance, firmante]
```

14.4 Definition of Ready (DoR) — antes de construir

```
- [ ] Valor y prioridad claros (tompo)
- [ ] Criterios de aceptación escritos (Gherkin/BDD)
- [ ] Diseño/contratos disponibles cuando aplica (eneida)
- [ ] Diseño de experiencia listo si hay UI (uxuifull)
- [ ] Dependencias identificadas (orquesta)
- [ ] Consideraciones de seguridad/privacidad señaladas (destroyer/sello)
```

14.5 Definition of Done (DoD) — antes de cerrar

```
- [ ] Correctitud revisada (certero)
- [ ] Pruebas pasando / cobertura acordada (amado)
- [ ] Performance dentro de budget (peakformance)
- [ ] Seguridad verificada / gates superados (destroyer + cluster)
- [ ] Desplegado y observable (baluarte)
- [ ] Documentado y trazable (docpower)
- [ ] Criterios de éxito verificados (tompo)
```

15. Cheat sheet — referencia rápida

15.1 Los 22 agentes en una línea

Agente	En una línea
`tompo`	El qué y el porqué: valor, backlog, prioridad.
`orquesta`	Quién/cuándo/en qué orden: coordina y desatasca.
`eneida`	Arquitectura, contratos y ADRs con trade-offs.
`uxuifull`	Experiencia: research, flujos, design system, accesibilidad.
`farola`	Frontend web.
`palma`	Mobile.
`motor`	Backend / APIs.
`baluarte`	CI/CD, IaC, SRE, SLOs, entrega y confiabilidad.
`peakformance`	Rendimiento: budgets, profiling, tuning cross-cutting.
`certero`	Correctitud: code review y debugging.
`amado`	Calidad: estrategia y ejecución de pruebas.
`docpower`	Documentación con forma profesional.
`andulfo`	Plataforma de datos / DBA / HA-DR.
`caudal`	Pipelines y movimiento de datos.
`brujula`	Capa semántica y métricas oficiales.
`prisma`	Ciencia de datos: modelos e insights.
`sysuniversal-ai`	IA en producción: serving, MLOps/LLMOps, RAG/agentes.
`destroyer`	Núcleo de seguridad: arquitectura, DevSecOps, cripto.
`cazador`	Ofensiva autorizada: pentest/red team.
`centinela`	Defensa: detección, hardening, DFIR/IR.
`sello`	GRC, ISO, privacidad, auditoría, riesgo.
`vector`	Seguridad de IA/LLM/agentes (MLSecOps).

15.2 Árbol de decisión de invocación

```
¿Sé qué necesito construir/lograr?
├─ No, necesito definir valor/prioridad ..... tompo
└─ Sí
   ├─ ¿Cruza más de un dominio o crea dependencias?
   │   ├─ Sí ..... orquesta (front
door)
   │   └─ No → ve al especialista del dominio:
   │       ├─ Arquitectura/decisión técnica ..... eneida
   │       ├─ Experiencia/UI diseño ..... uxuifull
   │       ├─ Frontend web ..... farola
   │       ├─ Mobile ..... palma
   │       ├─ Backend ..... motor
   │       ├─ Plataforma/CI-CD/SRE ..... baluarte
   │       ├─ Rendimiento ..... peakformance
   │       ├─ Revisión/bug ..... certero
   │       ├─ Pruebas/calidad ..... amado
   │       ├─ Documentación ..... docpower
   │       ├─ Datos (plataforma/pipeline/semántica) ... andulfo/caudal/brujula
   │       ├─ Ciencia de datos ..... prisma
   │       ├─ IA en producción ..... sysuniversal-ai
   │       └─ Seguridad ..... destroyer (+ cluster)
   └─ ¿Es sobre seguridad?
       ├─ Diseño/controles/secretos ..... destroyer
       ├─ Pentest (con RoE) ..... cazador
       ├─ Detección/incidente ..... centinela
       ├─ Cumplimiento/privacidad/ISO ..... sello
       └─ IA/LLM/agentes ..... vector
```

15.3 Orden canónico de una iniciativa

```
tompo → eneida → orquesta → uxuifull → [motor/farola/palma/cluster datos-IA]
      → certero + amado + peakformance + seguridad → baluarte → docpower →
retro
```

> ***Nota final:*** *este equipo rinde al máximo cuando respetas su gobierno: un dueño por decisión, handoffs explícitos, gates integrados y neutralidad tecnológica alimentada con tu contexto real. Úsalo como una organización de especialistas de élite, no como un chatbot único, y obtendrás resultados de nivel enterprise.*

Fin del manual — v1.0